

Data Handling: Import, Cleaning and Visualisation

Example Solution for Exercise/Workshop 2: Computer Code and Data Storage

Aurélien Sallin (lecturer) and Federica Mascolo (class tutor)

10/10/2024

Exercise solutions

Exercise A: Write a Sum Function

In the lecture on Programming with Data, we have discussed the most basic programming concepts in R. Among other things, you have learned how we can use for-loops to sum up numbers stored in a numeric vector.

1. Use the code example in the respective lecture notes and the following empty function construct to implement a function that takes a numeric vector as input and returns the sum of the vector's elements.

```
my_sum <- function(x){  
  
  # Initialize the total - we start at 0  
  total <- 0  
  
  for (num in x){  
    total <- total + num  
  }  
  
  return(total)  
}
```

2. Define the vector `numbers = c(1, 2, 3)`. Then, compare your function `my_sum()` with the `sum()` function (the pre-implemented sum function in R) when using `numbers` as the input argument: Do you get the same output for `my_sum(numbers)` and `sum(numbers)`?

We get the same result in both cases - our function seems to be working:

```
numbers = c(1,2,3)
```

```
my_sum(numbers)
```

```
## [1] 6
```

```
sum(numbers)
```

```
## [1] 6
```

Exercise B: Robustness and Warnings

1. Once you have implemented and successfully tested `my_sum()`, call your function with the following vector as input. What happens?

```
numbers2 <- c("1", "2", "3")
```

In this case We get an error, R will not compute the sum and complains about a non-numeric argument: 'Error in total + num : non-numeric argument to binary operator'

```
my_sum(numbers2)
```

2. Why do we get an error? Investigate by comparing the vector `numbers` from Exercise A with the new vector `numbers2`. Hint: the functions `str()` and `class()` might help you.

We can see that the numbers2 vector we provided does not contain numeric values but strings, i.e. a series of characters:

```
# Investigate the structures
str(numbers) # a diagnostic function
```

```
## num [1:3] 1 2 3
```

```
str(numbers2)
```

```
## chr [1:3] "1" "2" "3"
```

```
# Check the classes
```

```
class_numbers <- class(numbers)
```

```
class_numbers2 <- class(numbers2)
```

```
print(paste("Class of 'numbers':", class_numbers))
```

```
## [1] "Class of 'numbers': numeric"
```

```
print(paste("Class of 'numbers2':", class_numbers2))
```

```
## [1] "Class of 'numbers2': character"
```

3. Extend your `my_sum()` function with a control statement that checks whether the input is of the right class for the function to work (Hint: `class(x)!="numeric"`). The goal is to make the function more robust to wrong input. That is, if the input is wrong, issue a warning but don't break down with an error. Use the following code for the warning message:

```
warning("Wrong input! This function only accepts numeric or integer values!")
```

We can fix our code by adding an if-statement to make sure we get a warning in case the input is not numeric. The `is.numeric()` command will return `TRUE` if `x` is numeric (and also if `x` is an integer, hence it is more flexible than requiring `class == "numeric"` for example)

```
my_sum <- function(x){
  total <- 0

  # Check for the right input class
  if (is.numeric(x)) {
    for (num in x){
      total <- total + num
    }
  } else {
    warning("Wrong input! This function only accepts numeric or integer values!")
    total <- NA_real_ # if no sum can be computed, we return a missing value
  }

  return(total)
}
```

```
# Alternative
my_sum <- function(x) {
  if (!is.numeric(x)) {
    warning("Wrong input! This function only accepts numeric or integer values!")
  } else {
    return(sum(x))
  }
}
```

Note that we would not want to use our own function to sum stuff up, because on top of not being able to address missing entries (easy to fix, see first exercise), it is a lot slower:

```
# Creating vector with 100 million random (normally distributed) numbers
big_vector = rnorm(100000000)
# Time the built-in sum function
time_builtin_sum <- system.time({
  sum(big_vector)
})[3] # Extracting the elapsed time

# Time the custom my_sum function
time_my_sum <- system.time({
  my_sum(big_vector)
})[3] # Extracting the elapsed time

# Print the results
print(paste("Execution time of built-in sum:", time_builtin_sum, "seconds"))
```

```
## [1] "Execution time of built-in sum: 0.149999999999999 seconds"
```

```
print(paste("Execution time of custom my_sum:", time_my_sum, "seconds"))
```

```
## [1] "Execution time of custom my_sum: 0.140000000000001 seconds"
```

4. Test your enhanced sum function with the two input vectors `numbers` and `numbers2`.

We can see that we get a warning now when running the function on the second vector. Importantly, the function also returns a missing value for the sum, such that we are aware that no valid result is available. Specifically, the function should not return the value of 0 we initially set for the `total` in the function above, because that may cause a lot of issues if interpreted as a proper result.

```
my_sum(numbers)
```

```
## [1] 6
```

```
my_sum(numbers2)
```

```
## Warning in my_sum(numbers2): Wrong input! This function only accepts numeric or integer values!
```

Exercise C: use the sum function and lapply/loop/map on a list of vectors

1. `lapply()`: Part of base R's "apply" family of functions, which are used for functional programming.
2. **for loops**: A more traditional, imperative style of programming where you explicitly loop over elements.
3. `purrr::map()`: A function from the `purrr` package that provides a more modern and consistent interface for functional programming in R, part of the tidyverse collection of packages.

```
my_sum <- function(x){
```

```
  # Initialize the total - we start at 0
```

```

total <- 0

for (num in x){
  total <- total + num
}

return(total)
}

list_of_numbers <- list(
  c(1, 2, 3),
  c(4, 5, 6),
  c(7, 8, 9, 10)
)

# try to access the first element of the list: numeric vector
list_of_numbers[[1]]

```

```
## [1] 1 2 3
```

```

# lapply
sums_lapply <- lapply(list_of_numbers, my_sum)
print(sums_lapply)

```

```

## [[1]]
## [1] 6
##
## [[2]]
## [1] 15
##
## [[3]]
## [1] 34

```

```

# loop
# Initialize an empty list

# alternatives
sums_loop <- vector("list", length(list_of_numbers))
sums_loop <- rep(list(NULL), length(list_of_numbers))

```

```

sums_loop <- list()

for (i in 1:length(list_of_numbers)) {
  sums_loop[[i]] <- my_sum(list_of_numbers[[i]])
}

```

```
print(sums_loop)
```

```

## [[1]]
## [1] 6
##
## [[2]]

```

```
## [1] 15
##
## [[3]]
## [1] 34

# map:
# The map function transforms the input by applying a function to each element
# of a list or atomic vector and returning an object of the same length as the input.

library(purrr)
sums_map <- map(list_of_numbers, my_sum)
print(sums_map)

## [[1]]
## [1] 6
##
## [[2]]
## [1] 15
##
## [[3]]
## [1] 34
```

- `lapply()` takes a list (or vector) and applies a function to each element, returning a list. You can use it when you want to apply a function to each element of a list and get a list back. Is more concise and faster than loops.
- for loops explicitly iterate over each element of a list or vector, applying a function or operation. You can use it when you want to have more control of the operations. The logic is the same in other programming languages.
- `map()` is part of the `purrr` package (part of the tidyverse), which provides a consistent set of functions. It applies a function to each element of a list and returns a list. Variants like `map_dbl()`, `map_int()`, etc., return vectors of specific types. You can use it if you are working with tidyverse or you want to return specific types.

Exercise D: Data structures

Vectors, Factors, and Lists

1. Create a vector `student_ages` containing the ages of 5 students: 18, 21, 20, 22, 19.
 - print its length and type using `length()` and `typeof()` functions

```
# Create the vector
student_ages <- c(18, 21, 20, 22, 19)

# Print the length and type
length(student_ages) # Output: 5
```

```
## [1] 5
```

```
typeof(student_ages) # Output: "double" - numbers in the vector are stored as double-precision float
```

```
## [1] "double"
```

```
# as before you can also use:
class(student_ages)
```

```
## [1] "numeric"
```

```
str(student_ages)
```

```
## num [1:5] 18 21 20 22 19
```

2. Create a factor `student_grades` representing the grades of the same 5 students: "A", "B", "A", "C", "B".
 - print the factor levels, why do you use factors?

```
# Create the factor
student_grades <- factor(c("A", "B", "A", "C", "B"))

# Print the factor levels
levels(student_grades) # Output: "A" "B" "C"
```

```
## [1] "A" "B" "C"
```

Factors are useful here because grades are categorical data with a limited number of levels. Since factors store categorical data as integers, they are more memory-efficient than storing the same data as character strings. Statistical functions in R, such as `lm()` or `glm()`, automatically treat factors differently than numeric or character variables. Factors allow for easy inclusion of categorical variables in regression models, treating them as discrete groups.

When to use vectors: When your data does not have a fixed set of categories. When you need flexibility in adding, changing, or manipulating values frequently.

3. Create a list `student_info` that combines `student_ages`, `student_grades`, and another vector `student_names`.
 - access individual elements using both numeric and named indexing.

```
# Create the vector of student names
student_names <- c("Aurelien", "Federica", "Andrea", "Bob", "Eva")

# Create the list
student_info <- list(names = student_names, ages = student_ages, grades = student_grades)

# Access elements in the list
student_info$names
```

```
## [1] "Aurelien" "Federica" "Andrea"   "Bob"      "Eva"
```

```
student_info[[1]]
```

```
## [1] "Aurelien" "Federica" "Andrea"   "Bob"      "Eva"
```

4. What is the difference between a vector and a factor? Why do you use a list instead of a vector when dealing with different types of data?
 - **Difference between vector and factor:** A vector is a basic data structure for storing homogeneous data types (e.g., numbers, characters), whereas a factor is specifically designed for categorical data.
 - **Why use a list instead of a vector?:** A list can store elements of different types (e.g., vectors, factors), making it more flexible than a vector.

Matrices and Arrays

1. Create a matrix `scores_matrix` representing the scores of 3 students in 3 subjects.
 - give row and column names using `rownames()` and `colnames()`.
 - print the matrix and the score of a student that you prefer.

```

# Create the matrix
scores_matrix <- matrix(c(75, 82, 78, 88, 90, 92, 60, 65, 70), nrow = 3, byrow = TRUE)

# Set row and column names
rownames(scores_matrix) <- c("Student 1", "Student 2", "Student 3")
colnames(scores_matrix) <- c("Math", "English", "Science")

# Print the matrix
print(scores_matrix)

```

```

##           Math English Science
## Student 1    75      82      78
## Student 2    88      90      92
## Student 3    60      65      70

```

```

# Access the score of Student 2 in Science
scores_matrix["Student 2", "Science"]

```

```
## [1] 92
```

2. Create an array `student_array` to store the same data as in `scores_matrix` but for two semesters.
 - this 3-dimensional array, access “Student 3” scores in the first subject listed across both semesters.

```

# Create the array
student_array <- array(c(
  75, 82, 78, 88, 90, 92, 60, 65, 70, # Semester 1
  80, 84, 82, 91, 94, 93, 63, 68, 72 # Semester 2
), dim = c(3, 3, 2),
dimnames = list(
  c("Student 1", "Student 2", "Student 3"),
  c("Math", "English", "Science"),
  c("Semester 1", "Semester 2")
))

# Access "Student 3's" scores in "Math" across both semesters
student_array["Student 3", "Math", "Semester 1"]

```

```
## [1] 78
```

3. What are the differences between a matrix and an array? When would you use an array instead of a matrix?
 - A matrix is a 2-dimensional structure, while an array can have more than two dimensions.
 - Use an array when you need to store multi-dimensional data.

Data Frames, tibble and data.table

1. Create a data frame `students_df` combining all the data for 5 students: `student_names`, `student_ages`, `student_grades`, and another vector `attendance` containing their class attendance (in percentage): 85, 92, 78, 88, 90.
 - access the columns of attendance.
 - access the data for student 3.

```

# Create a data frame
students_df <- data.frame(
  names = student_names,
  ages = student_ages,
  grades = student_grades,

```

```

    attendance = c(85, 92, 78, 88, 90)
  )

  # Access a specific column (attendance, student_names...)
  students_df$names

## [1] "Aurelien" "Federica" "Andrea"   "Bob"      "Eva"

  # Access the data for "Andrea"
  students_df[students_df$names == "Andrea", "grades"]

## [1] A
## Levels: A B C

```

Difference between a list and a data frame: A list in R is a more general and flexible data structure than a data frame. A list is a one-dimensional collection of objects, and each object can be of any type and length. A data frame is a table-like structure in R, where each column represents a variable, and each row represents an observation (or case). Data frames are one of the most commonly used data structures in R for handling tabular data.

2. Why is a data frame more suitable for this kind of structured data than a matrix?
 - Data frames are more suitable for structured data because they can contain heterogeneous data types and offer better data manipulation functions.
3. Convert the existing students_df data frame into a tibble using the as_tibble() function.
 - name the tibble students_tibble and print the data frame and the tibble.

```

# Load the necessary package
library(tibble)

?tibble

## starting httpd help server ... done

# Convert to tibble
students_tibble <- as_tibble(students_df)

# Print the tibble and data frame to observe differences
print(students_df)      # Standard data frame print

##      names ages grades attendance
## 1 Aurelien  18      A           85
## 2 Federica  21      B           92
## 3 Andrea   20      A           78
## 4 Bob      22      C           88
## 5 Eva      19      B           90

print(students_tibble)  # More concise print for tibbles (not visible here because we have only 3 obs)

## # A tibble: 5 x 4
##   names    ages grades attendance
##   <chr>   <dbl> <fct>      <dbl>
## 1 Aurelien  18 A           85
## 2 Federica  21 B           92
## 3 Andrea   20 A           78
## 4 Bob      22 C           88
## 5 Eva      19 B           90

# Access a non-existent column in data frame and tibble
students_df$nonexistent # Output: NULL

```

```
## NULL
```

```
students_tibble$nonexistent # Output: NULL with a warning
```

```
## Warning: Unknown or uninitialised column: `nonexistent`.
```

```
## NULL
```

Tibbles provide a more readable print format and do not change types or convert strings to factors by default (you can for sure address this). Trying to access a non-existent column in a tibble gives a clear warning, unlike in data frames.

4. Create a data.table named students_dt from students_df using the as.data.table() function from the data.table package.

- convert students_df to a data.table
- select a column (ages), filter rows (grades equal to “A”)

```
# Load the necessary package  
library(data.table)
```

```
## Warning: package 'data.table' was built under R version 4.3.3
```

```
##
```

```
## Attaching package: 'data.table'
```

```
## The following object is masked from 'package:purrr':
```

```
##
```

```
##      transpose
```

```
# Convert to data.table
```

```
students_dt <- as.data.table(students_df)
```

```
# Selecting a column
```

```
print(students_dt[, ages]) # Output: Vector of ages
```

```
## [1] 18 21 20 22 19
```

```
# Filtering rows where grades are "A"
```

```
print(students_dt[grades == "A"]) # Output: Rows with grade "A"
```

```
##      names  ages grades attendance
```

```
##      <char> <num> <fctr>      <num>
```

```
## 1: Aurelien   18      A         85
```

```
## 2:  Andrea   20      A         78
```

```
# Adding a new column with the average attendance
```

```
students_dt[, average_attendance := mean(attendance)] # modifies the data table in place, meaning the  
print(students_dt)
```

```
##      names  ages grades attendance average_attendance
```

```
##      <char> <num> <fctr>      <num>          <num>
```

```
## 1: Aurelien   18      A         85           86.6
```

```
## 2: Federica   21      B         92           86.6
```

```
## 3:  Andrea   20      A         78           86.6
```

```
## 4:    Bob    22      C         88           86.6
```

```
## 5:    Eva    19      B         90           86.6
```

data.table has a more concise syntax for data manipulation tasks, such as filtering and adding new columns, and it is significantly faster, especially with large datasets

Additional Exercises for training

5. Use the output from the previous exercise. Compare the syntax of common data manipulation tasks (e.g., filtering, grouping, summarizing) between `data.frame`, `tibble`, and `data.table`.
 - Filter students with attendance greater than 85
 - Calculate mean age of students who passed

```
library(dplyr)

##
## Attaching package: 'dplyr'
##
## The following objects are masked from 'package:data.table':
##
##   between, first, last
##
## The following objects are masked from 'package:stats':
##
##   filter, lag
##
## The following objects are masked from 'package:base':
##
##   intersect, setdiff, setequal, union
# Filter students with attendance greater than 85

# Using data.frame
df_filtered <- students_df[students_df$attendance > 85, ]
# with subset
df_filtered <- subset(students_df, attendance > 85)

# Using tibble
tb_filtered <- students_tibble %>% filter(attendance > 85)

# Using data.table
dt_filtered <- students_dt[attendance > 85]
# as tidy verse
dt_filtered <- students_dt %>%
  .[attendance > 85]

# Calculate mean age of students who passed
# Using data.frame
mean(students_df$ages[students_df$grades > "C"])

## Warning in Ops.factor(students_df$grades, "C"): '>' not meaningful for factors
## [1] NA

# Using tibble
students_tibble %>% filter(grades > "C") %>% summarise(mean_age = mean(ages))

## Warning: There was 1 warning in `filter()`.
## i In argument: `grades > "C"`:
## Caused by warning in `Ops.factor()`:
## ! '>' not meaningful for factors
## # A tibble: 1 x 1
```

```
## mean_age
## <dbl>
## 1 NaN

# Using data.table
students_dt[grades > "C", mean(ages)]

## Warning in Ops.factor(grades, "C"): '>' not meaningful for factors
## [1] NaN

# Add a new column 'grade_numeric'
# Using data.frame
students_df$grade_numeric <- as.numeric(
  factor(students_df$grades, levels = c("A", "B", "C"), labels = c(4, 3, 2)))

# Using tibble
students_tibble <- students_tibble %>% mutate(
  grade_numeric = as.numeric(factor(grades, levels = c("A", "B", "C"), labels = c(4, 3, 2))))

# Using data.table
students_dt[, grade_numeric := as.numeric(
  factor(grades, levels = c("A", "B", "C"), labels = c(4, 3, 2)))]
```

Exercise E: Standard Deviation Function

1. During the lecture on Programming with Data we have step-by-step implemented our own R function to compute the mean. Now implement an R function called `my_sd` that computes the standard deviation, defined as: $SD = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (x_i - \bar{x})^2}$. Hint: revisit the math operators section in the notes on the first exercise session.

```
my_sd <- function(x) {
  n <- length(x)

  # First, check if n is greater than 1
  if(n <= 1) {
    warning("At least two numbers are required to compute standard deviation.")
    return(NA) # NA is returned if computation is not possible
  }

  mean_x <- mean(x)
  sum_squared_diff <- sum((x - mean_x)^2)
  sd <- sqrt(sum_squared_diff / (n - 1))

  return(sd)
}
```

2. Test your implementation by comparing its output with the already implemented `sd()` function.

```
# Create a test vector
set.seed(2023) # setting seed to reproduce the same results (note that results may still
               # vary across different devices and R-versions)
test_vector <- rnorm(10000) # a random vector of 10000 numbers with mean zero and sd of 1

# Compare the outputs of the custom function and the built-in function
custom_sd <- my_sd(test_vector)
builtin_sd <- sd(test_vector)
```

```
print(paste("Custom standard deviation:", custom_sd))

## [1] "Custom standard deviation: 0.996152686303493"

print(paste("Built-in standard deviation:", builtin_sd))

## [1] "Built-in standard deviation: 0.996152686303493"
```

Exercise F: Standard Error Function

Building on the function written in Exercise D, implement a function to compute (the estimate of) the sample mean standard error as defined here: $SE_{\bar{x}} = \frac{SD}{\sqrt{n}}$. Hint: the idea is to use your own implementation (in Exercise D) to compute SD as a first step of your standard error function.

Exercises D and E should illustrate how you can approach more complex problems by breaking them up in smaller problems and addressing each of these step-by-step. That is, when looking at the problem of computing $SE_{\bar{x}} = \frac{SD}{\sqrt{n}}$, you see that computing SD is a part of the overall problem, which can be addressed separately.

```
my_se <- function(x) {
  # Call the previously defined my_sd function to compute the standard deviation
  sd <- my_sd(x)

  n <- length(x)
  if(n <= 1) {
    warning("At least two numbers are required to compute standard error.")
    return(NA) # NA is returned if computation is not possible
  }

  se <- sd / sqrt(n)
  return(se)
}

# Test the function (base R has not built in function for this - we will test the
# final output below)
test_se <- my_se(test_vector)
print(paste("Standard error computed by my function:", test_se))

## [1] "Standard error computed by my function: 0.00996152686303493"
```

Exercise G: T-test

Implement a function for the one-sample t-test: $t = \frac{\bar{x} - \mu_0}{SE}$.

```
my_t_test <- function(x, mu_0) {
  # Calculate the sample mean
  sample_mean <- mean(x)

  # Compute standard error using the function from Exercise E
  se <- my_se(x)

  n <- length(x)
  if(n <= 1) {
    warning("At least two numbers are required to compute the t-statistic.")
    return(NA) # NA is returned if computation is not possible
  }
}
```

```

}

# Calculate the t-statistic
t_statistic <- (sample_mean - mu_0) / se

return(t_statistic)
}

# Testing against R

# Test the function with a sample vector and a test mean (mu_0)
mu_0_test <- 0 # This is the hypothesized mean (and the true mean of our data)

# Because our hypothesized value is correct, we should not reject, i.e.
# get a low t-statistic, well below the 5% significance threshold of +-1.96
t_test_result <- my_t_test(test_vector, mu_0_test)
print(paste("T-test result:", t_test_result))

## [1] "T-test result: -0.781507395524073"

# Now, use the built-in t-test function for comparison
builtin_t_test_result <- t.test(test_vector, mu = mu_0_test)$statistic

print(paste("Built-in T-test result:", builtin_t_test_result))

## [1] "Built-in T-test result: -0.781507395524073"

```